

Exercise 5: File Input/Output

Note: Before you start this exercise, if you already have Octave running and some variables defined, remove them with the “clear” command. Make a folder for today’s work in your Octave working directory, with a dated label in YYMMDD format. Remember to make notes in your research notebook as well!

1. File I/O

1.1 Writing to File

So far we have kept all of our information in our Octave workspaces. This information is stored in the computers RAM. RAM is the “working memory” of the computer. It is a good place to store information that we don’t need to record permanently, and that we want quick access to. However, every time we shut down Octave or turn off the computer we will lose all the information stored by our Octave programs. To store information more permanently, we need to write information to the computer’s hard drive. We will write programs that create new files and write information into the files. We will work from the template below.

```
%file created by XXX on YYMMDD
%tutorial exercise 5.1

clear;

%make vector
for i = 1:10
    test_vector(i) = 1 + (i-1)*2;
endfor

%save vector to file
filename = "test_file.bin";
file_ID = fopen(filename, "w");
fwrite(file_ID, test_vector);
fclose(file_ID);

%read vector from file
%file_ID = fopen(filename, "r");
% vector_read = fread(file_ID, Inf);
% vector_read2 = fread(file_ID, [1, Inf]);
%fclose(file_ID);
```

The first nine lines of code are a header, clearing the workspace, and then using a for loop to create a ten-element vector containing the odd numbers from 1 to 19. The next block of code, lines 11 to 15, creates a new file and then saves our vector into that file. In line 12 we declare a variable called `filename`, and then set it equal to the text `test_file.bin`. If you paste line 12 into your Command Window and run it, you will notice that Octave has stored the text `test_file.bin` as a 13-element vector, with each character in the vector stored as one element. We can use this like any other vector. For example, if we type `filename(11)` into the Command Window and hit enter, Octave will report that the value stored there is the character `b`.

In line 13 we use the `fopen` command to create/open a file. We give the `fopen` command two inputs, the file name and the one element text string `w`. The `fopen` command will check the current directory for a file with the specified name, open that file if it exists, and create a new file if none is present. Note that we are not required to call the variable that holds the name of the file `filename`, we just chose that name to make our code readable. The `fopen` command will take the first input inside the parentheses as the file name, no matter what we call it. We could also have skipped line 12 and just stated a filename inside the `fopen` command. Defining file names as variables is better choice for two reasons: it protects us from typos when using the file name, and it allows us to easily and accurately re-use this file name at any time during our program.

The second input for the `fopen` command defines the mode for working with the file. We chose `"w"`, which discards all current information in the file and allows us to write new information into the file. Other modes include: `"r"` for read-only access to a currently existing file, `"w+"` to discard current contents and grant both read and write access to a file, `"r+"` for read and write access without discarding current contents, `"a"` for write-only access appending new information at the end of a currently existing file, and `"a+"` for read and write access at the end of the file.

It may seem like a good idea to always use `"r+"` or `"a+"` modes to give yourself full access to all files all the time, but this can cause problems. Using read-only access when you want to read and write-only access when you want to write will allow the computer to double-check your work for you. If you try to write to a file that you have set up for read-only access, your program will crash and the computer will tell you exactly where and why the crash happened. This is a good thing, you want your programs to crash when they are not working properly. When

programs and data files get large and complex, it becomes extremely difficult to spot errors in reading and writing to files. You can protect yourself from these kinds of problems by defining your editing mode judiciously.

Note that line 13 isn't just the `fopen` command. We start with `file_ID =`, then use the `fopen` command. The first part of the command creates a variable named `file_ID`. You can think of this variable as holding the location and status of the opened file. It returns a value as well, but we don't use that value for anything most of the time. We use the `file_ID` variable to tell Octave which file we want to manipulate.

The `fwrite` command on line 14 writes our variable into the new file. It takes two inputs to accomplish this, first the file ID for the file we want to write to, and then the variable that we want to write into the file. Note that we use the `file_ID` variable here, and not the filename variable. Octave doesn't associate the filename variable with a particular file, it is just storing an array of characters there. The `file_ID` variable is the one associated with a specific file and editing mode.

Line 15 simply closes the file specified by `file_ID`. It is a good idea to close files as soon as you are done editing them, this prevents you from accidentally changing them later. If you try to refer to a closed file (i.e. if you use `file_ID` after you have closed the file), the program will crash. Again, this is good. We want broken programs to crash and for the computer to point out problems. We don't want our broken programs to do the wrong thing and report no problems. That kind of mistake is sneaky and hard to fix. Closing files as soon as you finish editing them is a great way to protect ourselves from this kind of problems. We can always open the file again later if we want to. You should also note that we used some formatting to make our program readable. We start a block of code with an `fopen` command, and end it with an `fclose` command. In between, we have indented lines that manipulate the file.

Put the code above into Octave and run it, if you haven't already. You should see a new file pop up in your File Browser, with the name `test_file.bin`. If you open the file in a text editor, you will not see the numbers stored in the vector written nicely for you in the file. You will see a few weird symbols. The computer has stored the information in what is called a binary format, which is very easy for the computer to read but much harder for us. The difference between readable

text and binary format has to do with bits and binary formatting, which we don't need to consider in detail. Just keep in mind that binary format is for computers to communicate with each other (or themselves), and text format is for human-readable files. We can use file suffixes like ".bin" or ".dat" for files that we want to be machine-readable. We should only use ".txt" when the file will contain human-readable text.

1.2 Reading from File

Uncomment lines 18, 19, and 21 (but not line 20), and re-run the program. There are a lot of similarities here to the previous block of code. Line 18 opens our file with read-only access. Line 19 uses the `fread` command, which takes two inputs. Once again the `file_ID` variable tells `fread` which file to work on. The `Inf` input tells `fread` to just read all the information present (i.e. infinity). Line 21 closes the file. If you look in the Workspace, you should see that the variable `vector_read` is now present as a 10x1 vector containing the same numbers as the variable `test_vector`.

Note that `test_vector` has a 1x10 structure, where `vector_read` has 10x1 structure. This change is caused by the structure we specified in our `fread` command. By specifying infinity as our second input, we told Octave to read all information present, and put each element in a different row. If we want to create multiple columns, we have to specify a different structure.

Comment out line 19 and uncomment line 20, then re-run the program. Now our `fread` command specifies some structure for the variable `vector_read2`. The "[1, Inf]" part specifies that Octave should read one element per row, and keep creating new columns until it runs out of data to read in the file. The variable `vector_read2` will have the same 1x10 structure as the variable `test_vector`. If we tell `fread` to look for more data than is actually present, *e.g.* by changing "[1, Inf]" into "[1, 50]", then `fread` will simply stop when it runs out of data. If we tell `fread` to look for less than the full amount of data in the file, it can execute that as well. If we change "[1, Inf]" to "[1, 5]", Octave will store the first five numbers in our sequence into the variable `vector_read2`.

2. Exercises (Complete before moving on, use more than one session if needed)

1. Revisit the Hydrogen energy level table. Define all information in the table as elements of a single, multi-dimensional variable. Next, write all this information to a binary file. Next, read the information back from the file and store it in a new variable. Finally, create a set of nested loops that compare the two original table variable and the new variable element-by-element, counts the number of elements that are the same and the number that are different, and reports back these numbers as output in the Command Window. (*HINT: You can use the function strcmp() to compare two strings. A string is another word for a vector of characters.*)
2. Modify the template code in this tutorial so that it stores the ten-element variable `test_vector` as a human-readable text file. You may need to spend some time reviewing chapter 14 of the online Octave manual, especially 14.2.16 Binary I/O (https://www.gnu.org/software/octave/doc/v4.0.3/index.html#SEC_Contents).
3. One reason to write information to file is that Octave only has access to a limited amount of RAM. The computer hard drive is much bigger. The only reason to chose RAM over hard drive is that RAM is much faster. Test Octave's RAM limits on the computer you are working with by defining variables (using loops or the `rand()` command). What is the maximum number of elements you can assign to a 1D vector? What about a square 2D matrix, or a symmetrical 3D tensor? Record these numbers in your notebook, you may want them later.
4. Can you create read/write functions for a 3D variable? It's trickier than you might think, especially the reading part. You may want to use some file I/O commands other than `fwrite` and `fread` (see Chapter 14 of the online manual for ideas.)